• Implement a secure environment for a database service

- Implement SQL Server-based options for authentication and authorization, and Azure options for securing Azure SQL databases.
- Explore encryption, firewalls, and advanced threat protection.

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL database, either on-premises on cloud-based
- Understanding of why security is a crucial part of database system planning
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal

Configure database authentication and authorization

Intermediate

Administrator

Azure

- Contrast authentication using Microsoft Entra ID, and SQL Server authentication.
- Implement various security principals and configure permissions.

Learning objectives

- After completing this module, you will be able to:
- Learn about authentication options for Azure SQL

Create various security principals

- Configure permissions within a SQL database
- Identify authentication and authorization failures

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL database, either on-premises or cloud-based
- Understanding of why security is a crucial part of database system planning
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level
- Experience creating and configuring resources using the Azure portal
- This module is part of these learning paths
- Implement a secure environment for a database service
- Describe authentication and identities
- Describe Security Principals
- Describe database and object permissions
- Identify authentication and authorization failures

• Exercise: Authorize Access to Azure SQL Database with Microsoft Entra ID

Database role	Definition
db_accessadmin	Allows users to create other users within the database. This role doesn't grant access to the schema of any of the tables, nor does it grant access to the data within the database.
db_backupoperator	Allows users to back up a database in a SQL Server or SQL Managed Instance. The role *db_backupoperator* doesn't confer any permissions in an Azure SQL Database.
db_datareader	Allows users to read from every table and view within the database.
db_datawriter	Allows users to INSERT, UPDATE, and DELETE data from every table and view within the database.
db_ddladmin	Allows users to create or modify objects within the database. Members of this role can change the definition of any object, of any type, but members of this role aren't granted access to read or write any data within the databases.
db_denydatareader	Users who need to be prevented from reading data from any object in the database, when those users have been

Database role	Definition
	granted rights through other roles or directly.
db_denydatawriter	Users who need to be prevented from writing data to any object in the database, when those users have been granted rights through other roles or directly.
db_securityadmin	Users who need to be able to grant access to other users within the database. Members of this role aren't granted access to the data within the database; however members of this role can grant themselves access to the tables within the database. Membership in this database role should be limited to only trusted users.
db_owner	Users who need administrative access to the database. Members of this role can perform any action within the database by default. However, unlike the actual database owner, who has the user name *dbo*, users in the db_owner role can be blocked from accessing data by placing them in other database roles, such as db_denydatareader, or by denying them access to objects. Membership in this database role should be limited to only trusted users.

Database Role	Definition
loginmanager	Allows members to create logins for the database server. This role is the equivalent of the securityadmin fixed server role in an on-premises Microsoft SQL Server.
dbmanager	Allows members to create and delete databases for the database server. This role is the equivalent of the dbcreator fixed server role in an on-premises Microsoft SQL Server.

Fixed server-level role	Definition
sysadmin	Allows its members to perform any activity on the server.
serveradmin	Allows its members to change server-wide configuration settings (for example Max Server Memory) and can shut down the server.
securityadmin	Allows its members to manage logins and their properties (for example, changing the password of a login). The members can also grant and revoke server and database level permissions. This role should be treated as being equivalent to the sysadmin role.
processadmin	Allows its members to kill processes running inside of SQL Server.
setupadmin	Allows its members to add and remove linked servers using T-SQL.
bulkadmin	Allows its members to run the BULK INSERT T-SQL statement.
diskadmin	Allows its members to have the ability to manage backup devices in SQL Server.
dbcreator	Allows its members to create, restore, alter, and drop any database.
public	Every SQL Server login belongs to the public user role. Unlike the other fixed server roles, permissions can be granted, denied, or revoked from the public role.

Permission	Definition
SELECT	Allows the user to view the data within the object (table or view). When denied, the user is prevented from viewing the data within the object.
INSERT	Allows the user to insert data into the object. When denied, the user is prevented from inserting data into the object.
UPDATE	Allows the user the update data within the object. When denied, the user is prevented from updating data in the object.
DELETE	Allows the user to delete data within the object. When denied, the user is prevented from deleting data from the object.

Permission	Definition
CONTROL	Grants all rights to the objects. It allows the user who has this permission to perform any action they wish against the object, including deleting the object.
REFERENCES	Grants the user the ability to view the foreign keys on the object.
TAKE OWNERSHIP	Allows the user the ability to take ownership of the object.
VIEW CHANGE	Allows the user to view the change tracking setting for the object.

Database role	Definition
TRACKING	
VIEW DEFINITION	Allows the user to view the definition of the object.

Permission	Definition
ALTER	Grants the user the ability to change the definition of the object.
CONTROL	Grants the user all rights to the object.
EXECUTE	Grants the user the ability to execute the object.
VIEW CHANGE TRACKING	Allows the user to view the change tracking setting for the object.
VIEW DEFINITION	Allows the user to view the definition of the object.

Error	Action
Login failures	Look for any outages during the time when the application reported the errors at Microsoft Azure Service Dashboard.
Database reaches resource limits	Monitor your database's compute and storage resources carefully, and take action when it reaches its resource limits to prevent transient failures.
Extended authentication failures	File an Azure support request through the Azure portal if your application encounters connectivity error for longer than 60 seconds or if it occurs more than once in a given day.

Summary

- **1 minute**
- Azure SQL offerings provide several authentication and authorization options that differ from those in SQL Server.
- This is because Azure SQL Database and Azure SQL Managed Instance rely on Microsoft Entra ID instead of Windows Server Active Directory.
- This module explores best practices for granting permissions and the various permissions available within a database.
- It also delves into the concept of least privilege.
- While built-in roles in SQL Server and other database engines offer broad security privileges, many applications require more granular security on database objects.
- Now that you've reviewed this module, you should be able to:
- Explain authentication options for Azure SQL offerings
- **Describe security principals**
- **Explain object permissions**
- Identify authentication and authorization failures

Protect data in-transit and at rest

Intermediate

Data Analyst

Data Engineer

Database Administrator

Azure

Azure SQL Database

SQL Server

- Explore encryption options available within Azure SQL, including firewall rules, Always Encrypted, and Transport Layer Security.
- Understand how SQL Injection works.

Learning objectives

- After completing this module, you will be able to:

- Understand the data encryption options available in the various platforms
- **Implement object level encryption**
- Understand the difference between database and server firewall rules for Azure SQL Database
- Explore Always Encrypted with secure enclaves

Prerequisites

- Ability to use tools for running queries against a Microsoft SQL database, either on-premises or cloud-based
- Understanding of why security is a crucial part of database system planning
- Ability to write code in the SQL language, particularly the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal

Introduction

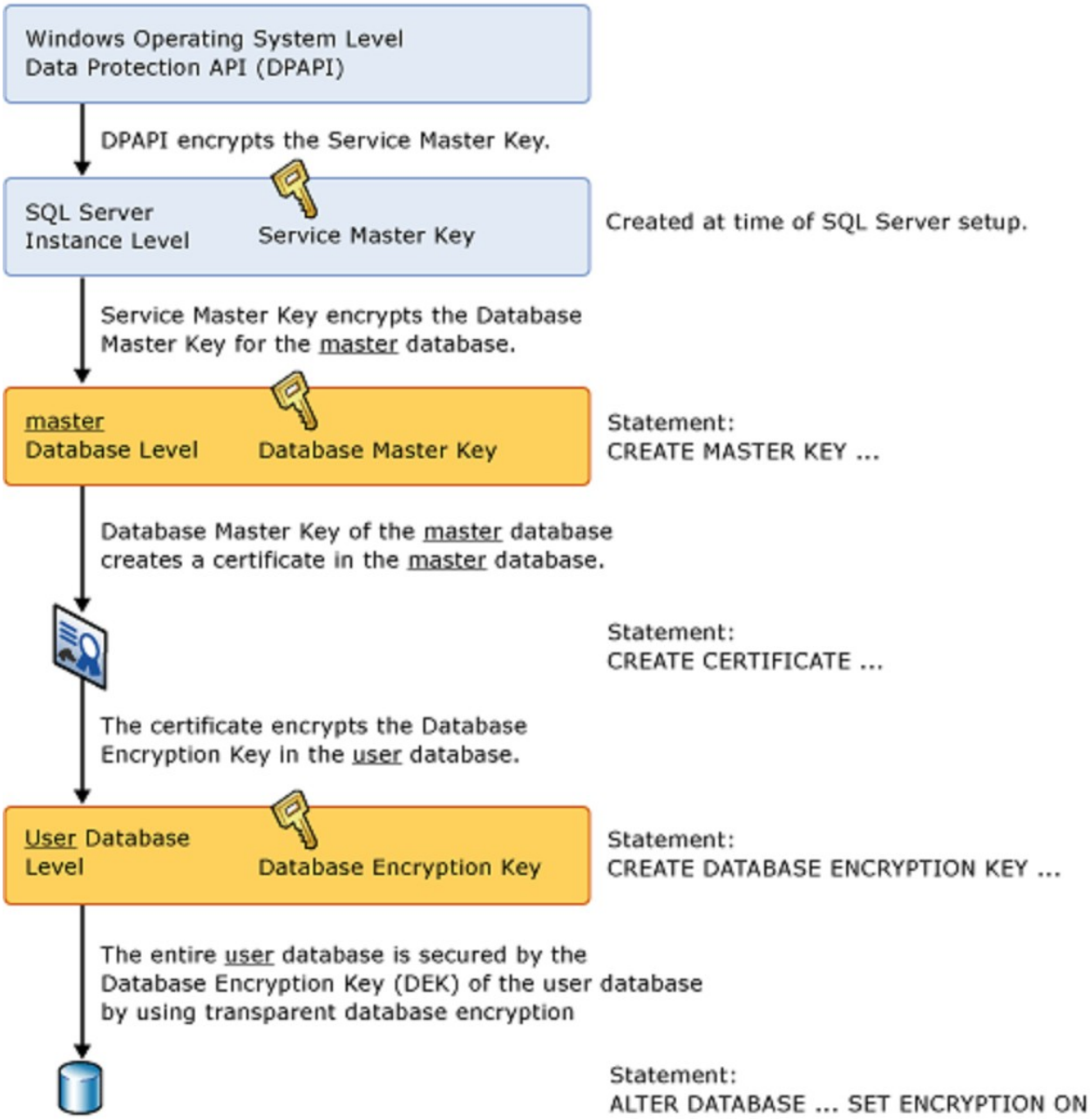
- **1 minute**
- Explore the encryption options available within Microsoft SQL Server, Azure SQL Database, and Azure SQL Managed Instance.
- Each of the various platforms supports different database encryption options.
- Students explore these data encryption options and how to configure them.
- Consider the following three scenarios when evaluating data encryption methods.

Scenario	Definition
Data at rest	Encrypting it while it's on file storage.
Data in transit	Encrypting it while it travels through private or public network communication channels.
Data in use	Encrypting it while it's in RAM or CPU caches.

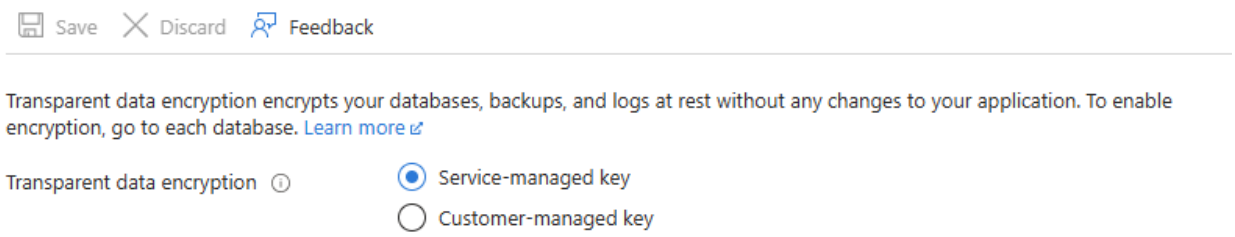
Explore Transparent Data Encryption

- Microsoft SQL Server’s Transparent Data Encryption (TDE) encrypts all data within a target database at the page level.
- Data is encrypted as it's written to the data page on disk and decrypted when read into memory, resulting in all data pages on disk being encrypted.
- TDE doesn't encrypt data at the table or column level.
- Anyone with the appropriate permissions can read, copy, and share the data.
- Encryption at rest protects against restoring a backup to an unsecured server or copying database and transaction log files to another unsecured server.
- No decryption occurs during the backup operation.
- TDE protects data at rest and complies with various industry laws, regulations, and guidelines.
- It allows software developers to encrypt data using AES encryption algorithms without changing existing applications.

Transparent Database Encryption Architecture



- Databases created in Azure SQL Database after May 2017 have TDE enabled automatically.
- Databases created before May 2017 need TDE to be manually enabled.
- For Azure SQL Managed Instance, TDE is enabled by default for databases created after February 2019.
- Databases created before February 2019 need TDE to be manually enabled.
- To enable TDE in an Azure SQL Database, edit the database in the Azure portal.
  From the Transparent data encryption pane, select to enable data encryption.



- By default, databases in Azure SQL Database are encrypted using a Microsoft-provided certificate (service-managed key).
- Azure also offers a Bring Your Own Key (BYOK) option, allowing you to use a customer-managed key created by your company and uploaded to Azure Key Vault.
- If the customer-managed key is removed from Azure, database connections are closed, and access to the database are denied.
- Enabling TDE within a Microsoft SQL Server database is an easy process as only a few T-SQL commands are required.
- This process involves the following steps:

```
Set a master key within the master database using the CREATE MASTER KEY ENCRYPTION command.
Create a certificate in the master database using the CREATE CERTIFICATE command.
Create a database encryption key within the database using the CREATE DATABASE ENCRYPTION KEY command.
```

- Enable the encryption key using the ALTER DATABASE command.

SQL

```
USE master;
GO

CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<your-pwd>';
GO

CREATE CERTIFICATE MyServerCert
WITH SUBJECT = 'TDEDemo_Certificate';
GO

USE [TDE_Demo];
GO

CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256 ENCRYPTION BY SERVER CERTIFICATE MyServerCert;
GO

ALTER DATABASE TDE_Demo SET ENCRYPTION ON;
GO
```

- Once TDE is enabled, it takes time to encrypt the database as each page must be read, encrypted, and written back to disk.
- The larger the database, the longer this process takes.
- This background process runs at a low priority to avoid overloading the system's I/O or CPU.
- The certificate used by TDE must be manually backed up and stored securely.
- SQL Server integrates with Enterprise Key Managers (EKMs) like Azure Key Vault to manage encryption keys.

```
Managing the certificate is crucial because if it's lost and the database needs to be restored from a backup, the restore fails as the database can't be read.
```

Note

- To use TDE with databases in an Always On Availability Group, the certificate used to encrypt the database must be backed up and restored to the other servers within the Availability Group that will be hosting copies of the database.

- Customer-managed keys
- You can alternately use BYOK and take advantage of an Azure key vault.
- The advantages of using customer-managed keys are:
- Full and granular control over usage and management of the TDE protector
- **Transparency of the TDE protector usage**
- Ability to implement separation of duties in the management of keys and data within the organization
- The key vault administrator can revoke key access permissions to make encrypted database inaccessible
- **Central management of keys in AKV**
- Greater trust from your end customers because AKV is designed so that Microsoft can't see or extract encryption keys
- You can also take advantage of using a user-assigned managed identity (UMI) with customer-managed keys for TDE, which:
- Enables the ability to preauthorize key vault access for Azure SQL logical servers by creating a user-assigned managed identity and granting it access to key vault, even before the server or database are created.
- Allows creation of an Azure SQL logical server with TDE and CMK enabled.
- Enables the same user-assigned managed identity to be assigned to multiple servers, eliminating the need to individually turn on system-assigned managed identity for each Azure SQL logical server and providing it access to key vault.
- Provides the capability to enforce CMK at server creation time with an available built-in Azure policy.
- Automatic key rotation are introduced for customer-managed keys using TDE.
- When enabled, the server continuously checks the key vault for any new versions of the key being used as the TDE protector.

  If a new version of the key is detected, the TDE protector on the server is automatically rotated to the latest key version within 24 hours.

Azure disk encryption

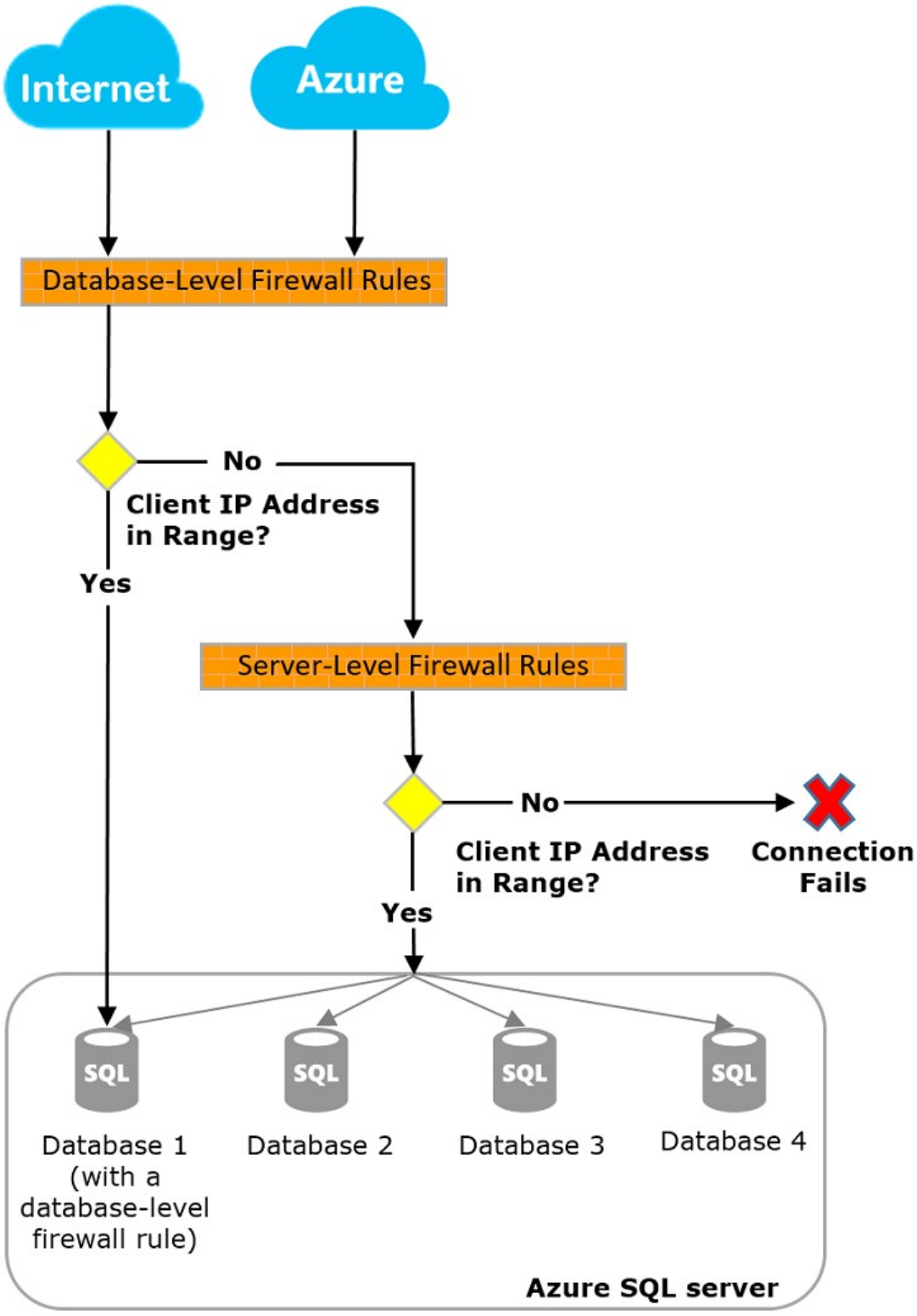
- In addition to these SQL Server security features, Azure VMs include an extra layer of security, Azure Disk Encryption—a feature that helps protect and safeguard data and meet organization and compliance commitments.

  If you're using TDE, your data is protected by multiple layers of encryption with Azure Disk Encryption.

- Configure server and database firewall rules
- Firewalls prevent unauthorized users from accessing protected resources.
- Each Azure SQL Database maps to a public IP address hosted by Microsoft.
- In each Azure region, one or more public IP addresses allow you to reach your database gateway, which then directs you to your database.

How firewall works

As shown in the following diagram, connection attempts coming from the internet and Azure must go through the firewall before they reach your server or database.



- Azure provides built-in firewalls to limit access in order to protect your database and your data.
- In Azure SQL Database there are two distinct sets of firewall rules: server-level firewall rules and database-level firewall rules.
- Server-level firewall rules
- Both server and database level firewalls use IP Address rules instead of SQL Server Logins, and allow all users at the same public IP Address to access the SQL Server.
- For most companies, this is their outbound IP address.
- Server-level firewalls are configured to allow users to connect to all databases on the server.

Database level firewalls are used to grant or block specific IP Addresses from accessing specific databases.

Server level firewall rules can be configured using the Azure portal or using the sp_set_firewall_rule stored procedure from within the master database.

Note

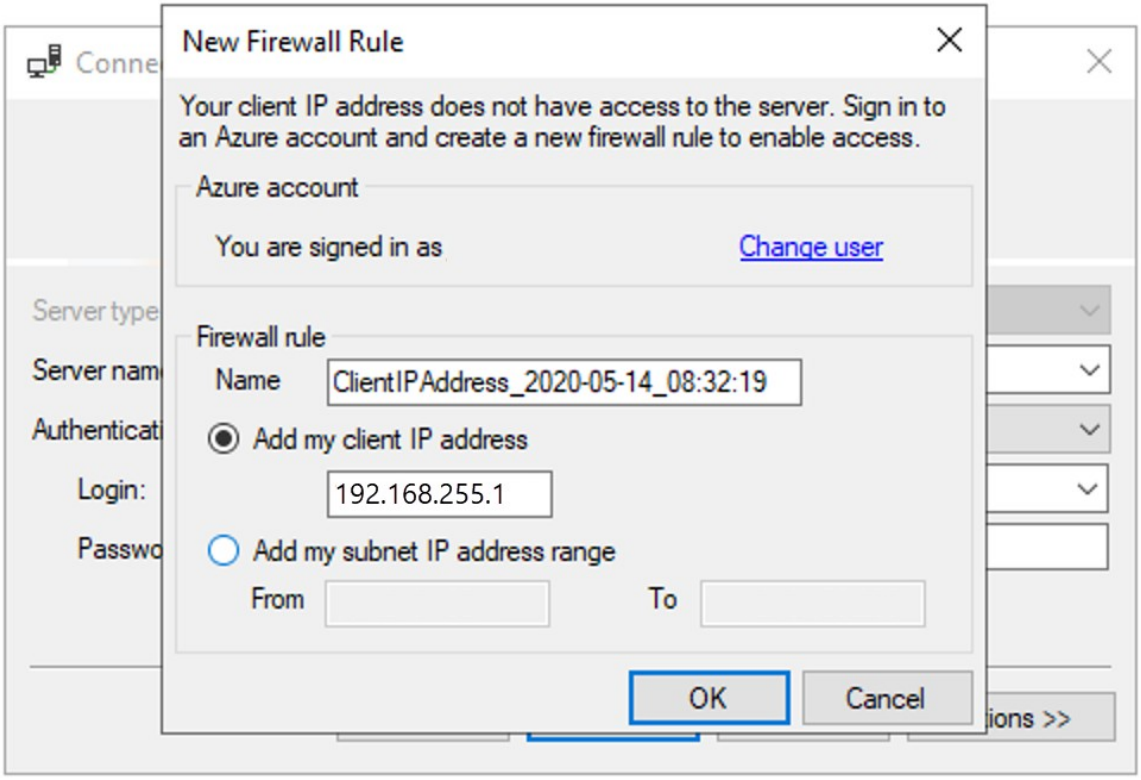
- The Allow Azure Services and resources to access this server server setting counts as a single firewall rule when enabled.

Database-level firewall rules

Database-level firewall rules are configured through T-SQL only using the sp_set_database_firewall_rule stored procedure from within the user database.

- Upon connection, Azure SQL Database first checks for a database-level firewall rule corresponding to the database name specified in the connection string.
- If no such rule exists, the firewall then checks the server-level IP firewall rules.
- Server-level IP firewall rules apply to all databases on the server.

If a matching rule is found at either level, the connection is established.
If neither exist and the user is connecting through SQL Server Management Studio, they'll be prompted to create a firewall rule.



Virtual network endpoints

Virtual network endpoints allow traffic from a specific Azure Virtual Network.

- These rules apply at the server level, not the database level.
- Additionally, the service endpoint applies to only one region, which is the underlying endpoint's region.
- Another important consideration is that the virtual network connecting to the Azure SQL Database must have outbound access to the database's public IP address.
- This can be configured using service tags for Azure SQL Database.
- Private link
- The Private Link feature allows you to connect to Azure SQL Database and other PaaS offerings using a private endpoint.
- A private endpoint allows for a connection to your Azure SQL Database to go entirely over the Azure backbone network and not over the public internet.
- This feature provides a private IP address on your Virtual Network.
- Another feature of private link is that it allows for Azure Express Route connections through that circuit.

- Private link offers several benefits including cross-region private connectivity and protection against data leakage by only allowing connections to specific resources.

- Explain object encryption and secure enclaves
- In addition to supporting encryption at rest, SQL Server supports encrypting data within columns using Always Encrypted.
- Once data is encrypted, the application accessing the database must have the correct certificate in order to view the plain text values of the data.

Always Encrypted

Always Encrypted allows for the encryption of data within the client application, protecting sensitive data from malware and high-privileged users, such as Database Administrators (DBAs), server admins, cloud admins, or those who manage the data but should have no access.

- This encryption happens automatically based on the settings within the Microsoft SQL Server database, which tell the application what the encryption settings on the database column are.
- The following table provides some scenarios for Always Encrypted usage:

Scenario	Definition
Client and data on-premises	For scenarios where you need to protect your on-premises database from high-privileged users, for example, external vendors managing SQL Server.
Client on-premises with data in Azure	In this scenario, to ensure Microsoft cloud administrators have no access to the data, Always Encrypted keys are stored in key store hosted on-premises, for SQL Database or SQL Server running in a virtual machine on Microsoft Azure.
Client and Data in Azure	In this scenario, the environment is fully hosted on Azure. While Always Encrypted doesn't completely isolate data from cloud administrators, the customer still benefits from the fact that the data is encrypted in the database.

- Always Encrypted is based on a master encryption key and a column encryption key.
- Having both keys allows each column to be encrypted using a different encryption key for maximum data protection.
- Always Encrypted has various key stores that can store the certificate used by encryption.
- Here's an example of enabling Always Encrypted.
- You can see that NationalIDNumber and BirthDate columns are both in plain text.

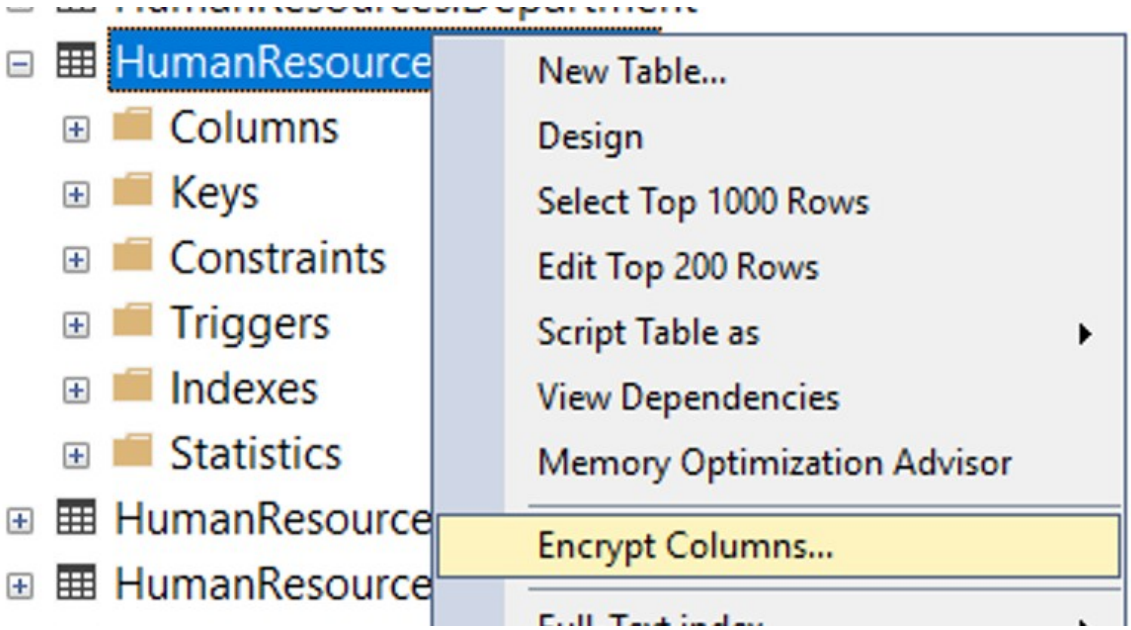
```
SELECT
      [NationalIDNumber]
     ,[LoginID]
     ,[OrganizationNode]
     ,[OrganizationLevel]
     ,[JobTitle]
     ,[BirthDate]
  FROM [AdventureWorks2017].[HumanResources].[Employee]
```

100 %

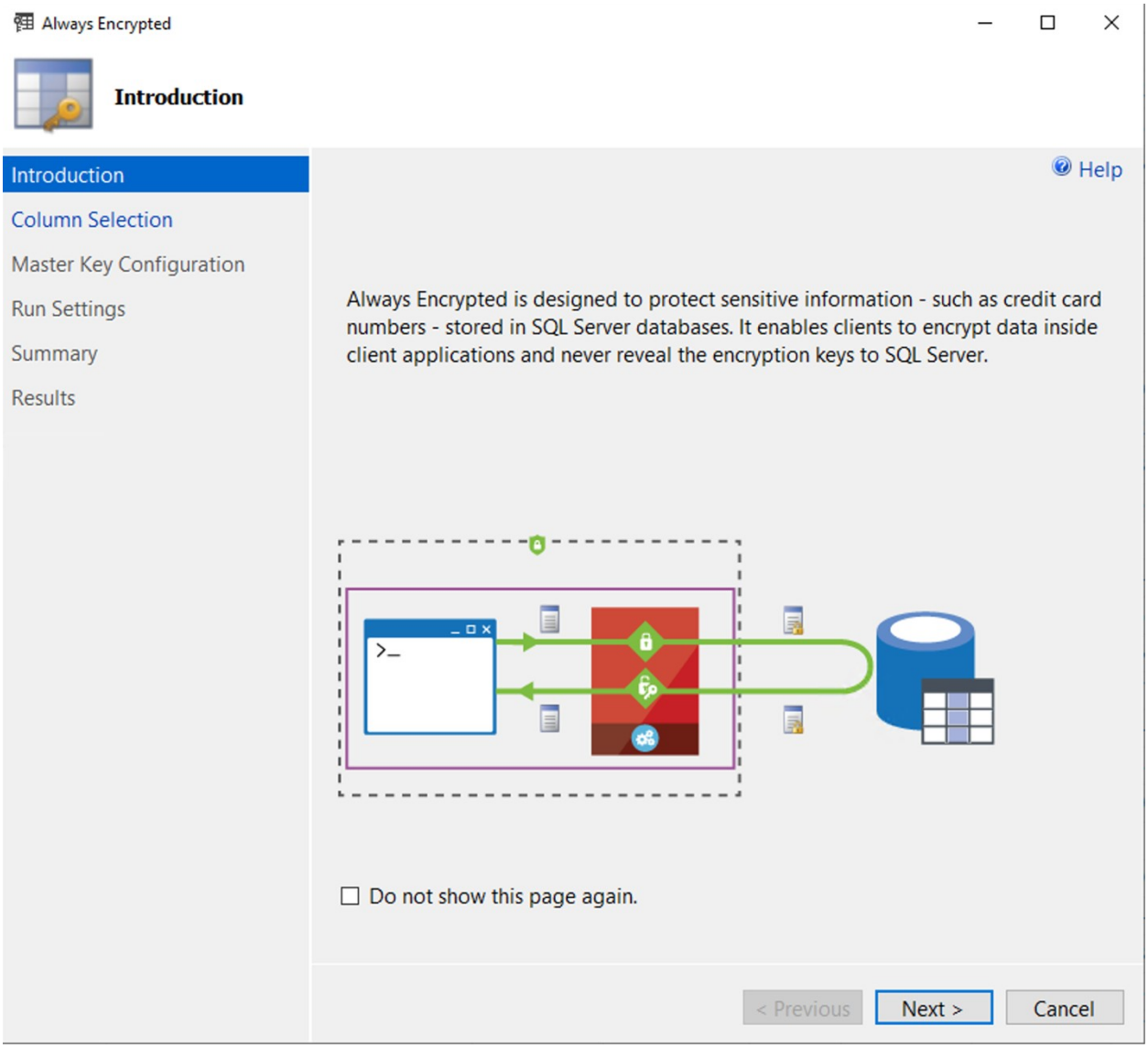
Results | Messages

	NationalIDNumber	LoginID	OrganizationNode	OrganizationLevel	JobTitle	BirthDate
1	295847284	adventure-works\ken0	NULL	NULL	Chief Executive Officer	1969-01-29
2	245797967	adventure-works\terri0	0x58	1	Vice President of Engineering	1971-08-01
3	509647174	adventure-works\roberto0	0x5AC0	2	Engineering Manager	1974-11-12
4	112457891	adventure-works\rob0	0x5AD6	3	Senior Tool Designer	1974-12-23
5	695256908	adventure-works\gail0	0x5ADA	3	Design Engineer	1952-09-27
6	998320692	adventure-works\jossef0	0x5ADE	3	Design Engineer	1959-03-11
7	134969118	adventure-works\dylan0	0x5AE1	3	Research and Development Manager	1987-02-24
8	811994146	adventure-works\diane1	0x5AE158	4	Research and Development Engineer	1986-06-05

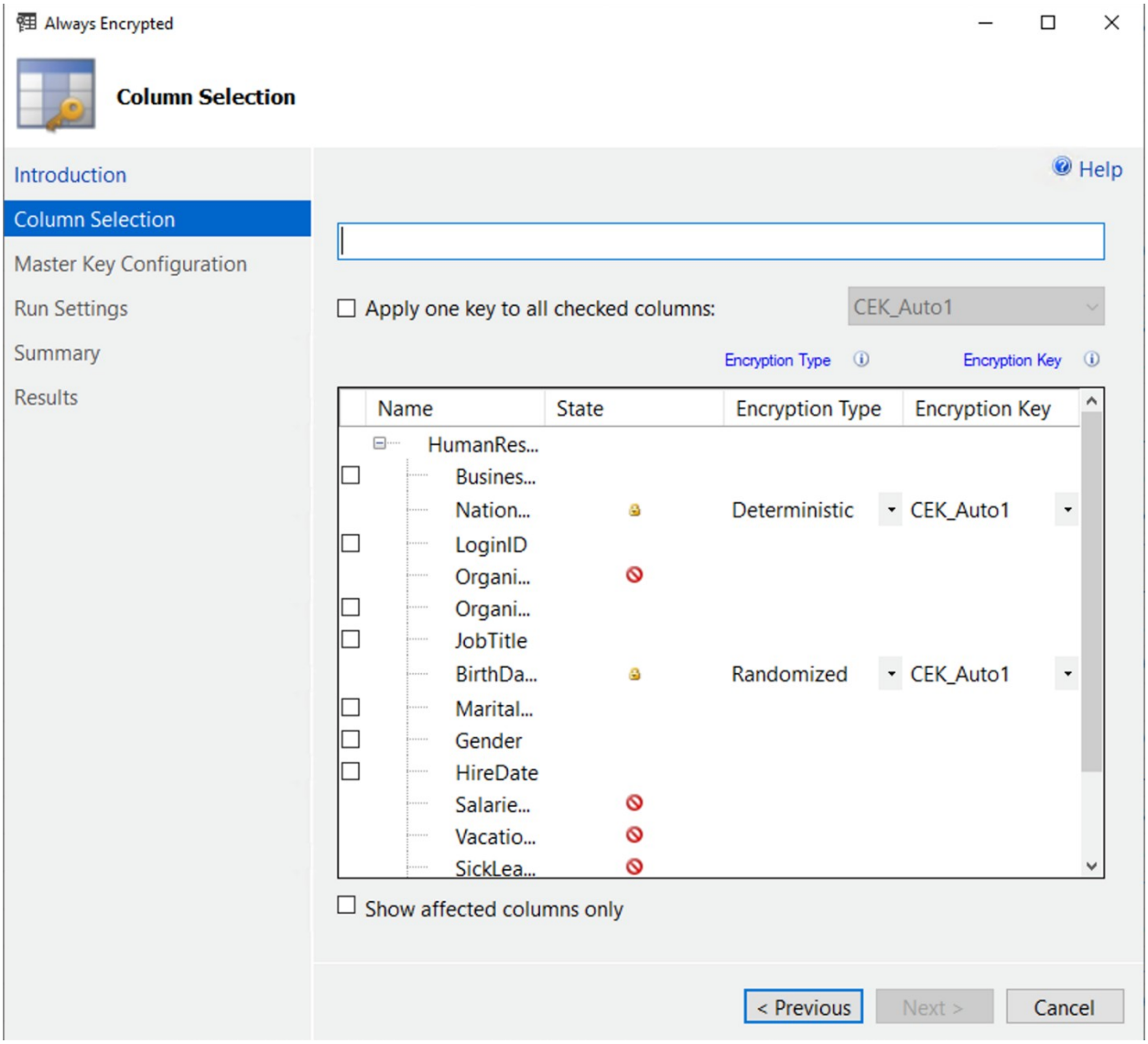
- The next few images show you how we can encrypt both of these columns using Always Encrypted.
- The encryption could be done using T-SQL, but in this example, we use the wizard from SQL Server Management Studio.
- You can reach the wizard by right-clicking on the table name in Object Explorer as shown below.



When you select Encrypt Columns..., the wizard launches.

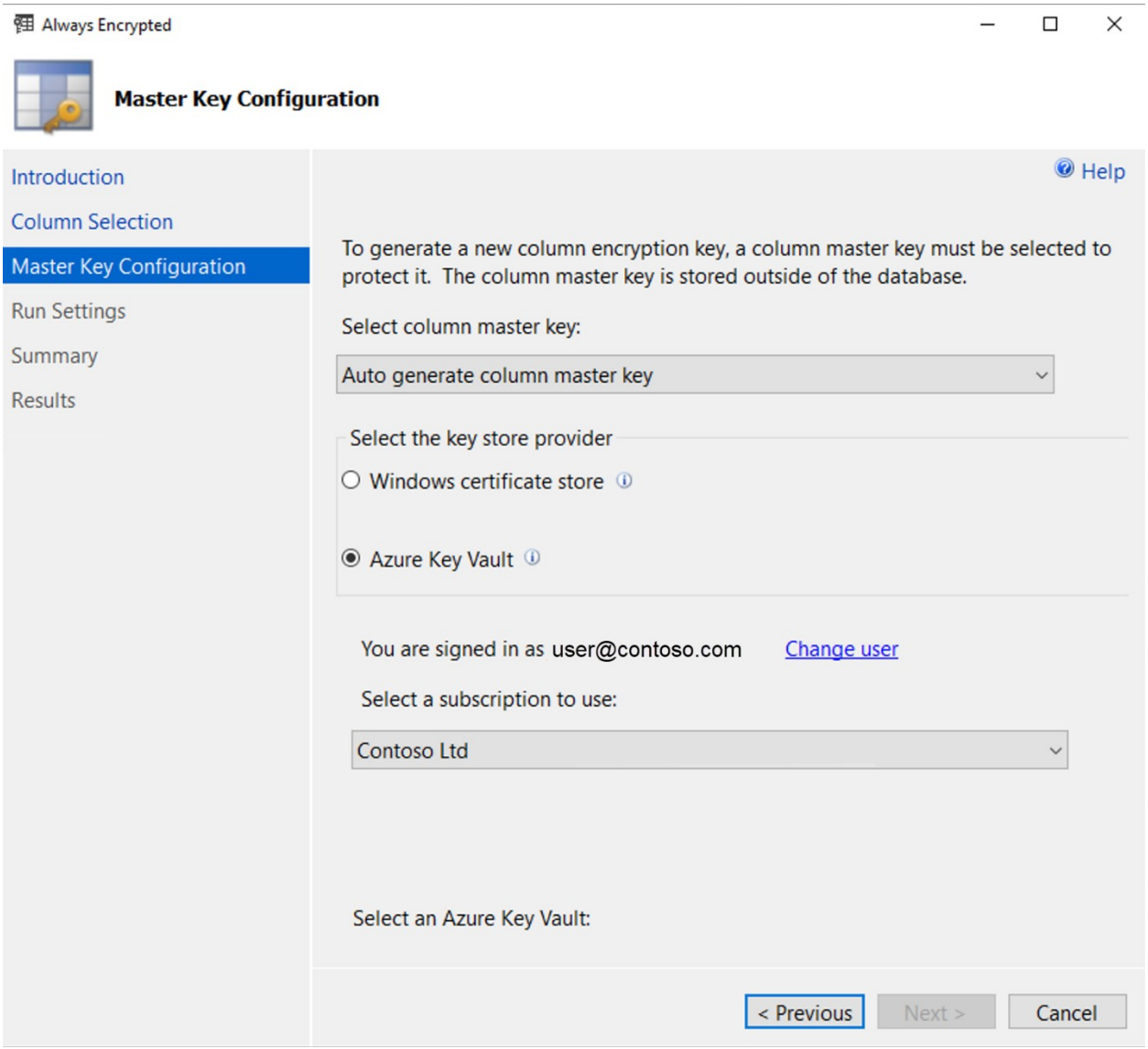


Select Next to choose the columns you want to encrypt.



- There are two different types of encryption specified.
- The NationalIDNumber column is encrypted with Deterministic encryption, and the BirthDate column is encrypted using Randomized encryption.
- Randomized encryption is more secure than deterministic encryption but comes with limitations.
- Once a column is created, you can't change its encryption type.
- It's recommended to use randomized encryption for columns with a few well-known distinct values that could be guessed by someone with access to the encrypted data, such as a three-digit credit card verification code.
- Always Encrypted with randomized encryption is limited because the same value is encrypted differently each time.
- This means you can only return these columns in your results.
- In contrast, deterministic encryption always encodes a value the same way, allowing for comparisons using equality and inequality operators, as well as joins, grouping, and indexing.
- Also, the wizard generates a column encryption key, which performs the data encryption.
- Each encrypted column can have its own key, or you can use the same key for multiple columns.

After identifying the columns you're encrypting, you can select Next and you'll see the Master Key Configuration screen:



In this screen, you create the column master key, which is used to encrypt the column encryption keys.

- You can supply your own key, if you're using T-SQL to encrypt the columns.
- This key must be stored in a key store such as the Windows Certificate Store, Azure Key Vault, or a hardware security module.

  The database engine never stores the column master key, and only contains the metadata about where it's stored.
  Not storing the master key protects data access from users who have full access to the database.
- For the highest level of security, the key should be stored within a third-party key store such as Azure Key Vault.
- Never generate the keys on the server hosting your database, as the key could potentially be extracted from memory on that server.
- In the example, the key is being stored in Azure Key Vault.
- On the next screen, the wizard will provide you with the option to either finish the encryption process now, or to generate a PowerShell script.
- Once you complete the process, the data appears as encrypted to anyone querying the data without the key.

```
select * from HumanResources.Employee
```

100 %

Results | Messages

	BusinessEntityID	NationalIDNumber	LoginID	OrganizationNode	OrganizationLevel	JobTitle	BirthDate
1	1	0x01C3979CCA373497E69A372F3F0A091E096D1ABC60C573BE...	adventure-works\ken0	NULL	NULL	Chief Executive Officer	0x01C39E5FEDE5C8965A81D512F3C5EA6E6D97AF13B40AA6CF...
2	2	0x01B8B50F9C9599EAEB1BD653F01FCE7321671F6F0FA0B22D3...	adventure-works\terri0	0x58	1	Vice President of Engineering	0x014DEF44661EA605D07B955A8283525523A57206970670C7D...
3	3	0x01BDD49E6DF84DFB95FD5C42EC511A86A6C5FCA45B8C63F...	adventure-works\roberto0	0x5AC0	2	Engineering Manager	0x01CE6DAD98D6B28AF76EA565565615A662A025F16185BCD...
4	4	0x01E8634C71B297A23C9AC247F682C2BE00976CCFA5265F836...	adventure-works\rob0	0x5AD6	3	Senior Tool Designer	0x0117B23F362D932E0380FAF28A9EBF7B59C68AA21840AB725...
5	5	0x01B49F37A7628AB9093EC44AAA071FFAF3193B2F820A5B5C5...	adventure-works\gail0	0x5ADA	3	Design Engineer	0x01DFA4BEB32386DA7F39C8BEA1CF34CCF71A2F38F3F185519...
6	6	0x01D2760AEC79ED9659F70DD3AC8E26232DD645B4D3B2E1F...	adventure-works\jossef0	0x5ADE	3	Design Engineer	0x0186FF5BAD106F4ACDFF47194FB0484615A7D812959B5D492...
7	7	0x019FD96C3C09F255FAA6F789D105CAA8D41E49D3B972E0E3...	adventure-works\dylan0	0x5AE1	3	Research and Development Manager	0x0134262E465846A90A7B2E134A22092AC1A582829D3A19F38...
8	8	0x0192802B7AAAD3386A78AA9071FD9F8395C6BB2538320F03...	adventure-works\diane1	0x5AE158	4	Research and Development Engineer	0x0153027957F29425CF8949093C8A08742812BC8C71E01BC6C1...
9	9	0x01B2A71B39816D3BE16023B774F92BBC72060A69E9E00E1E4...	adventure-works\gigi0	0x5AE168	4	Research and Development Engineer	0x01F458C1E6E5C0D493F13AD2617844C2BEDAE057E074E5713...
10	10	0x01975B1459B2F73ED3E8A4253DB05CF0AA48DA3613083CC4...	adventure-works\michael6	0x5AE178	4	Research and Development Manager	0x01FA2B454B98604F48B6F2985B4D9D022305E38CF4FA6B4C2...
11	11	0x01553EF331044B52447480DA345E586285ACDF7FE0163E25F...	adventure-works\ovidiu0	0x5AE3	3	Senior Tool Designer	0x01EC3C91F0168EFB338026F3E0643B3ECD5B7AA0CA23F65BA...
12	12	0x01041122F34858C6171448A3FBEEF2BA97EC148BC8E3E119E...	adventure-works\thierry0	0x5AE358	4	Tool Designer	0x01D946A0CF8B619E8C212BB8C9ED95DDDE8B7758DFD0467...
13	13	0x015C53D73AC0DFCE594D1CF72DE9837A0F11A9A317405B62...	adventure-works\janice0	0x5AE368	4	Tool Designer	0x018D56101A359B7680CCD9E00FD13013C39473746CFD98C7...
14	14	0x015EADCC6AC88F6C12844545074303726F104A26DD95C90B...	adventure-works\michael8	0x5AE5	3	Senior Design Engineer	0x0101929CAC7B3F4A8A7F3FE1C729C034EBF582D5D357DF11...
15	15	0x0120328DDE84AC51BB081396151CE1191F473EE04FD36AF1...	adventure-works\sharon0	0x5AE7	3	Design Engineer	0x018761024DFE8DE4CA8DFAB0E04494A2707B0413E27862B7...

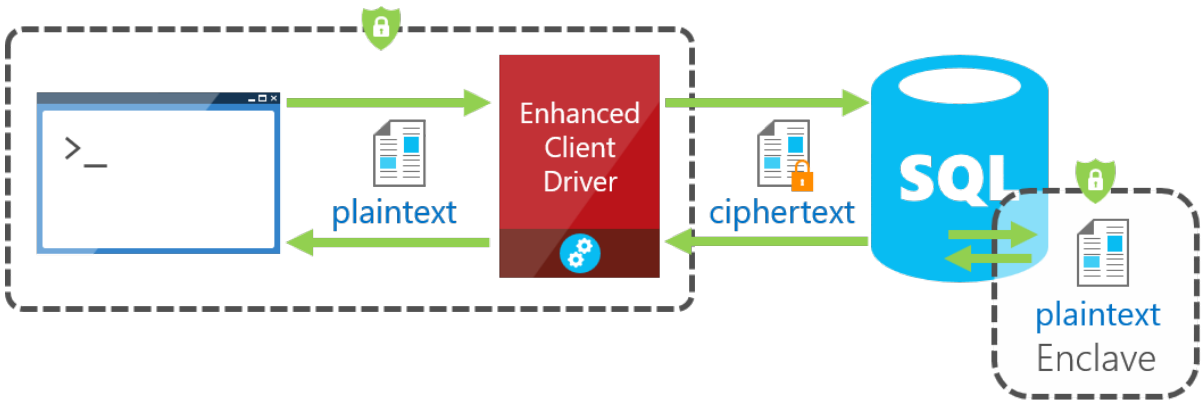
In order to decrypt data from an Always Encrypted column, your application needs an Always Encrypted driver to connect to the database, followed by the following actions:

- The application has access to the key store where the Always Encrypted keys are stored
- **The application then retrieves the data**
- Data that is written back to the database is encrypted at the client through the driver
- In addition to the driver, the application's connection string needs to have the setting Column Encryption Setting=enabled provided.
- This setting causes a metadata lookup to be made for each column that is used by the application.

- Note
- To minimize metadata lookups, the application needs to update the SqlCommandColumnEncryptionSetting on the SqlConnection objects within the .NET application.
- These settings must be set for each database query that the application submits.

- Secure enclaves
- Always Encrypted supports a feature called secure enclaves, which allows more robust querying of encrypted data.
- A secure enclave is a secured region of memory within the SQL Server process that acts as a trusted execution environment for processing encrypted data.
- This enclave appears as a black box to SQL Server, and it isn't possible to view any data or code, even with a debugger.

The image shows the architecture of this process:



- Always Encrypted with secure enclaves also addresses some of the limitations of Randomized encryption, which enables pattern matching, comparison operations, and indexing on columns using this encryption type.

Enable encrypted connections

- It's possible to encrypt data that is transmitted across a network between an instance of SQL Server and a client application with Transport Layer Security (TLS).
- The TLS encryption is performed within the protocol layer and is available to all supported SQL Server and Azure SQL database services.

- Certificates
- You must run SQL Server Configuration Manager with a local administrator account in order to install certificates for use by SQL Server.
- Furthermore, the certificate must satisfy the following conditions:

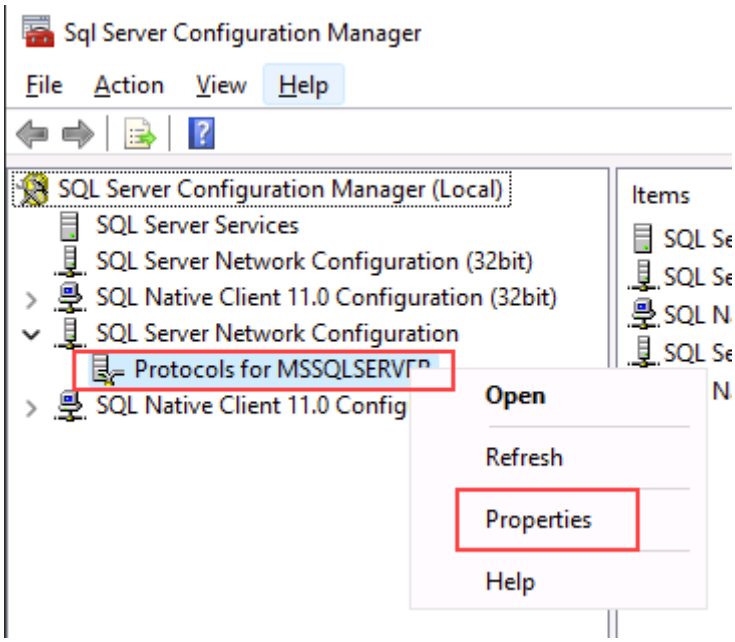
- The certificate must be located in the local computer certificate store or the current user certificate store.
- The SQL Server service account must have permission to access the certificate.
- The certificate must be within a valid period.

Note

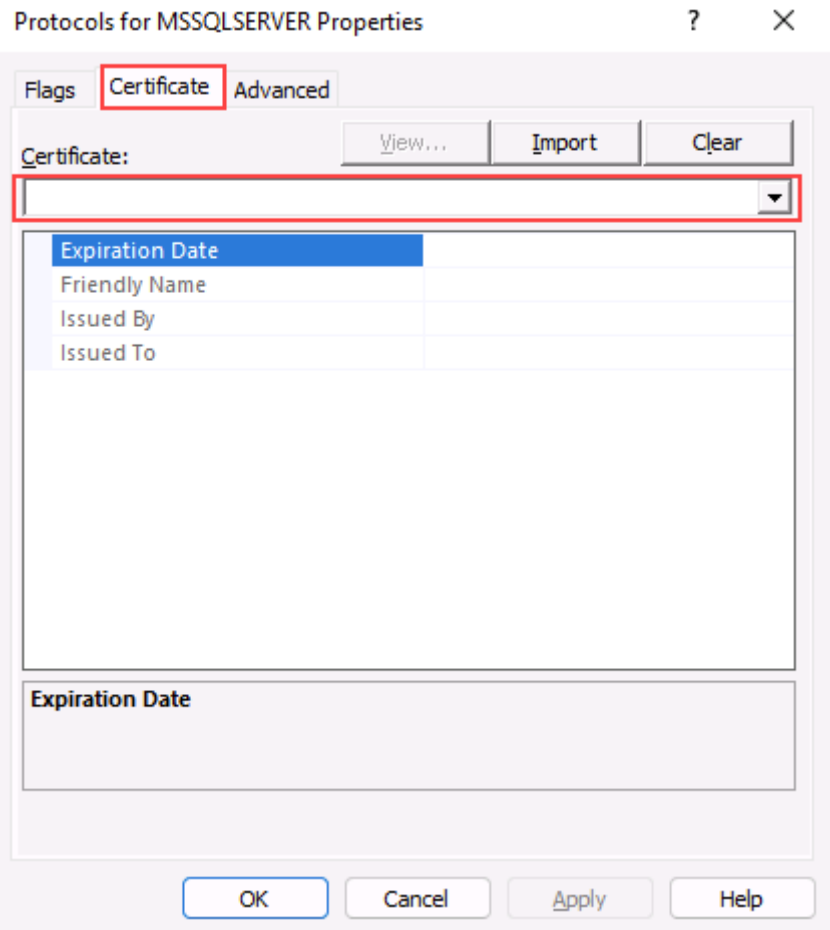
If the correct access isn't provided, restarting SQL Server service fails.

- For a complete list of requirements when installing a TLS certificate, see Enable encrypted connections to the Database Engine.
- Configure SQL Server instance
- You can configure a SQL Server instance to use encrypted connections by following these steps:

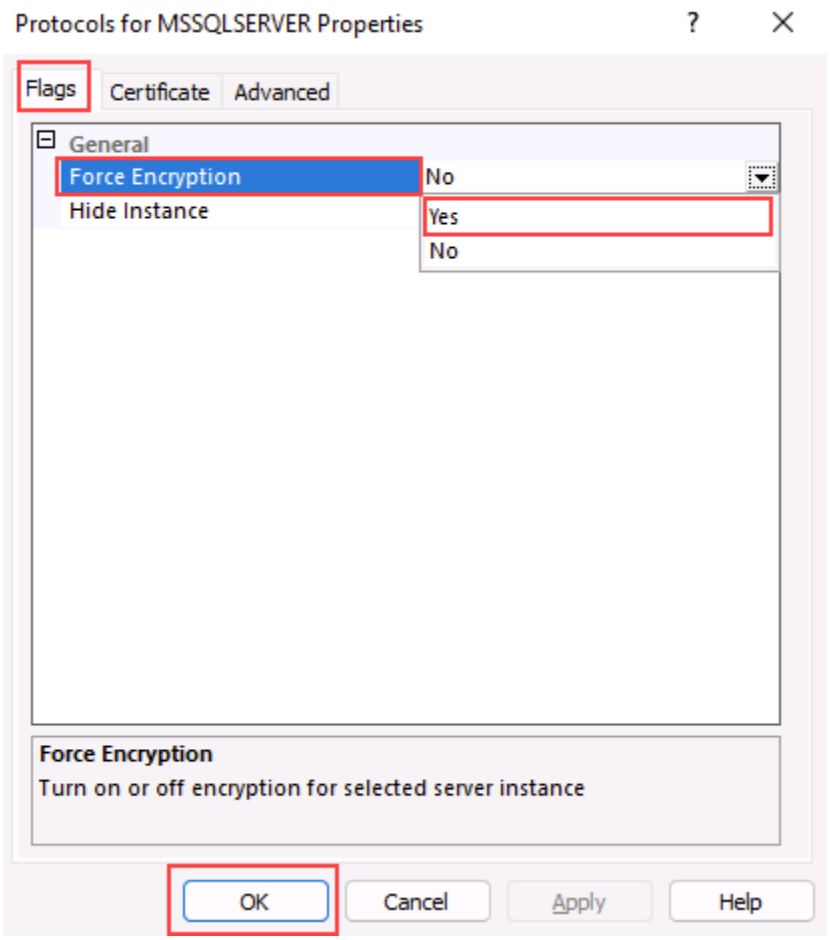
On the SQL Server Configuration Manager, expand SQL Server Network Configuration, right-click Protocols for <server instance>, and then select Properties.



From the Protocols for <server instance> Properties dialog box, select the Certificate tab, then select the certificate from the Certificate drop-down.

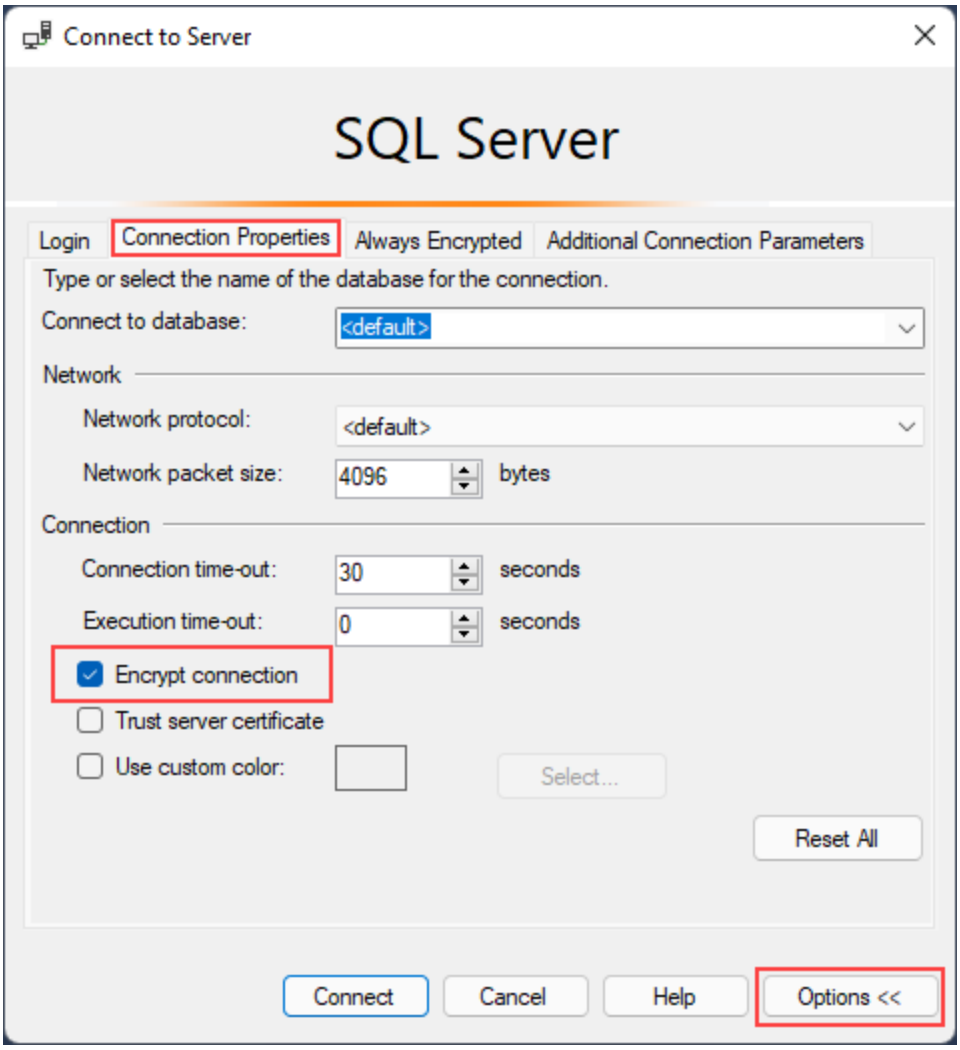


On the Flags tab, in the ForceEncryption property, select Yes, and then select OK.



- Restart the SQL Server service.
- Once the necessary configuration is in place, you can test the connection through SQL Server Management Studio:

  In the Connect to Server dialog box, complete the connection information, and then select Options.
  On the Connection Properties tab, select Encrypt connection, and then Connect.



- All steps must have been executed correctly for you to be able to authenticate through SQL Server Management Studio using TLS.

Describe SQL injection

- SQL injection is a common method used for data breaches.
- The attack involves appending an SQL command to a form field in a web or application front end, usually through a website, with the intent of breaking the original SQL script and executing the injected SQL script.
- This often occurs when dynamically generated SQL is used within the client application.
- The primary cause of SQL injection attacks is poor coding practices in both the client application and database stored procedures.
- While many developers adopt better practices, SQL injection remains a significant problem due to the prevalence of legacy applications and newer applications built by developers who didn't prioritize SQL injection prevention.
- As an example, assume that a front-end web application creates a dynamic SQL statement as follows:

SQL

```
SELECT * FROM Orders WHERE OrderId=25
```

- This T-SQL is created when the user goes to the sales order history portion of the company's website and enters 25 into the form field for the order ID number.
- However, suppose the user enters more than just an ID number, for example "25; DELETE FROM Orders;"
- In that case, the query sent to your database would be as follows:

SQL

```
SELECT * FROM Orders WHERE OrderID=25; DELETE FROM Orders;
```

- The way the query in the above example works is that the SQL database is told via the semicolon ";" that the statement has ended and that there's another statement that should be run.

The database then processes the next statement as instructed, which would result in the deletion of all rows from the Orders table.

The initial SELECT query is run as normal without any errors being generated.

- However, when you look at the Orders table, you don't see any rows.
- The second query in the batch, which deletes all the rows, was also executed.
- One technique used to prevent SQL injection attacks is to inspect the text of the parameters, or values entered into the form fields, looking for various keywords.
- However, this solution only provides minimum protection as there are many, many ways to force these attacks to work.
- Some of these injection techniques include passing in binary data, having the database engine convert the binary data back to a text string, and then executing the string.
- You can run the following T-SQL code to see an example of this scenario.

SQL

```
DECLARE @v VARCHAR(255)

SELECT @v = cast(0x73705F68656C706462 AS VARCHAR(255))

EXEC (@v)
```

When accepting data from a user, whether a customer or an employee, it's crucial to validate that the input matches the expected data type to prevent SQL injection attacks.

If a number is expected, the client application should verify that the input is indeed a number.

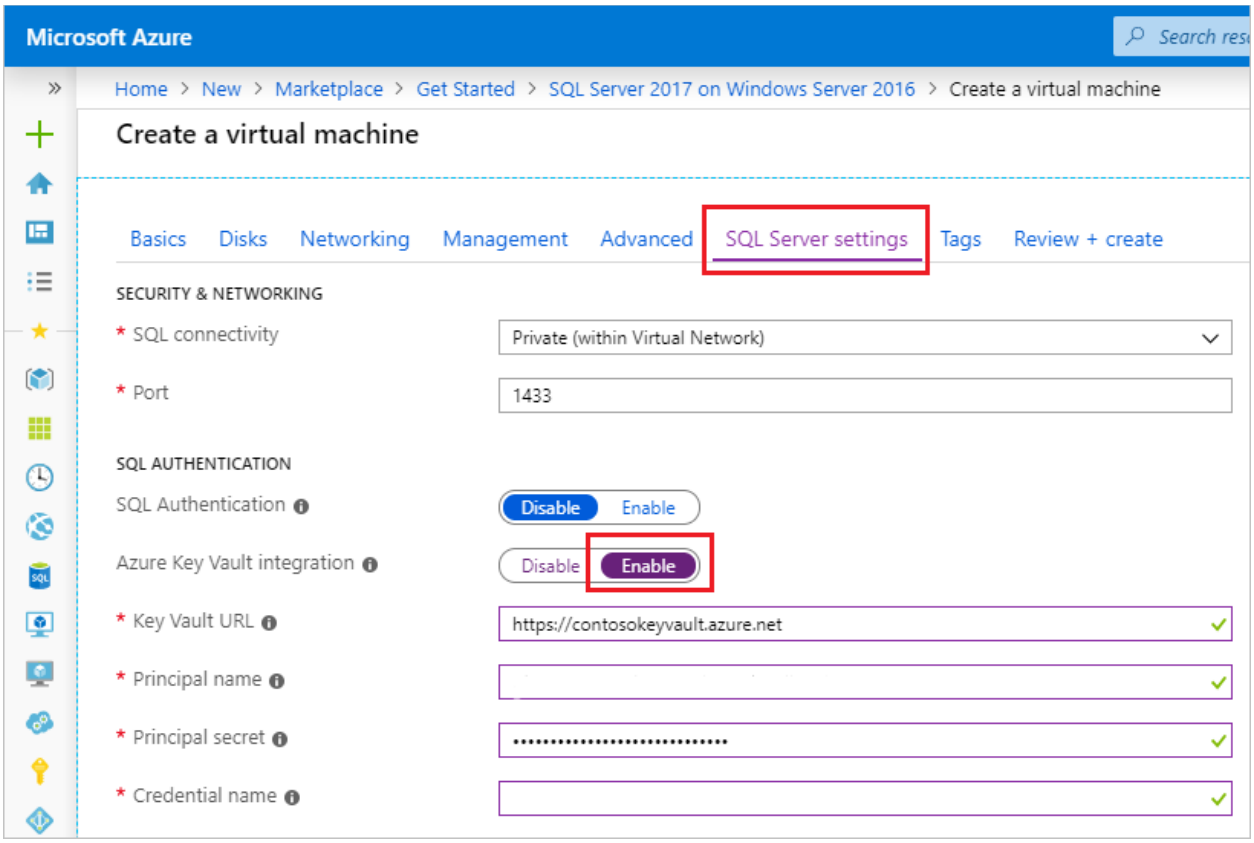
If a text string is expected, ensure it is of the correct length and doesn't contain any binary data.

- The client application should validate all user input.
- Validation can be done by either informing the user of the issue and allowing them to correct it, or by gracefully handling the error to ensure no commands are sent to the database or file system.
- While fixing your application code should always be the priority, there may be cases where it's not possible.
- In such cases, Advanced Threat Protection can offer an extra layer of security for your sensitive data.

Understand Azure Key Vault

- Azure Key Vault is a tool used for storing and accessing secrets.
- Whether they be passwords, certificates or keys, Key Vault acts as a secure area for those secrets to be accessed in a secure fashion, typically programmatically.
- Key Vault data has its own RBAC policies, separate from the Azure subscription.
- This means someone who is in the role of subscription admin won't have access to the Key Vault unless explicitly granted.
- SQL Server, either within an Azure Virtual Machine or on-premises, supports using Azure Key Vault to store certificates for features such as Transparent Data Encryption, Backup Encryption, or Always Encrypted.

While this configuration is complex in an on-premises environment, it's easily managed when using SQL Server on Azure Virtual Machine, as shown in the following image.



- In order to configure the Azure Key Vault integration, you need to set the Key Vault URL, the Principal name, the Principal secret, and the name of the credential.
- This task can be done at the virtual machine creation or to an existing VM.
- Configuring SQL Server to connect to Azure Key Vault first requires creating a normal SQL Server login within the instance.
- Next a Credential needs to be created and mapped to the login.
- For the identity of the credential, the name of the key vault should be used.
- For the secret of the credential, use the application ID from Azure Key Vault.
- Once the credential is created, an asymmetric key can be created within the Azure Key Vault.
- An asymmetric key can then be created within the SQL Server database.
- The key in database can be mapped to the Azure Key Vault asymmetric key using the CREATE ASYMMETRIC KEY command with the FROM PROVIDER syntax.
- Once an asymmetric key is created within the database, that key can be used for TDE, or Backup Encryption or Always Encrypted.

Exercise: Configure a server-based firewall rule using the Azure portal

- In this exercise, you'll learn how to configure a server-based firewall rule using the Azure portal.
- Note
- To complete this exercise, you'll need an Azure subscription.
- Launch the exercise and follow the instructions.

Summary

- **1 minute**
- This module explored the encryption options available within Microsoft SQL Server and Azure SQL.
- Each of the various platforms supports different database encryption options.
- You also explored these data encryption options and how to configure them.
- Now that you've reviewed this module, you should be able to:
- Understand the difference between server and database firewall rules in Azure SQL Database

- Understand how Always Encrypted is used to protect data in transit
- Understand the role of Azure Key Vault in Transparent Data Encryption
- Explain how to enable encrypted connections
- Implement compliance controls for sensitive data

Intermediate

Data Analyst

Data Engineer

Database Administrator

Azure

Azure SQL Database

SQL Server

- Explore data classification capabilities and degrees of confidentiality.
- Implement security options to maintain private data safe, including Azure SQL auditing, Microsoft Defender for SQL, row-level security, Dynamic Data Masking and Ledger.

Learning objectives

- After completing this module, you will be able to:
- Plan and implement data classification in Azure SQL Database
- Understand and configure row-level security and dynamic data masking
- Understand the usage of Microsoft Defender for SQL
- **Explore how Ledger works**

Prerequisites

- Ability to write code in the SQL language, particular the Microsoft T-SQL dialect, at a basic level.
- Experience creating and configuring resources using the Azure portal.
- This module is part of these learning paths
- Implement a secure environment for a database service
- Explore data classification
- Explore server and database audit
- Implement Dynamic Data Masking
- Implement Row Level security
- Understand Microsoft Defender for SQL
- Explore Ledger
- Implement Microsoft Purview

• Exercise: Enable Microsoft Defender for SQL and Data Classification

Action group	Definition
BATCH_COMPLETED_GROUP	Audits all the queries and stored procedures executed against the database.
SUCCESSFUL_DATABASE_AUTHENTICATION_GROUP	This indicates that a principal succeed to log into the database.
FAILED_DATABASE_AUTHENTICATION_GROUP	This indicates that a principal failed to log into the database.

Masking function	Definition	T-SQL example
Default	Masks the data in the column without exposing any part of the values to the user. The user would see XXXX for string values, 0 for numbers, and 01.01.1900 for date values.	ALTER TABLE [Customer] ALTER COLUMN Address ADD MASKED WITH (FUNCTION = 'default()')
Credit card	Masks all but the final four characters, allowing users to view the final four digits. This masking can be useful for customer service agents who need to view the last four digits of a credit card number but who don't need to see the entire number. The data is shown in the usual format of a credit card number XXXX-XXXX-	ALTER TABLE [Customer] ALTER COLUMN Address ADD MASKED WITH (FUNCTION = 'partial(0,"XXXX-XXXX-XXXX-",4)')

Action group	Definition	
	XXXX-1234.	
Email	Only the first letter and the trailing domain suffix aren't masked; for example, “aXXX@XXXXXXX.com”	ALTER TABLE [Customer] ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()')
Number	This masking format should be used on numeric columns. It shows a random number as the masked value instead of the actual value. With each query, a different number is displayed.	ALTER TABLE [Customer] ALTER COLUMN [Month] ADD MASKED WITH (FUNCTION = 'random(1, 12)')
Custom string	This option allows text to be masked with any value, and to display a custom number of characters at either end of the masked value. If the length of the value being masked is equal to or less than the number of characters which the mask specifies are to be displayed, then only the masked characters are displayed.	ALTER TABLE [Customer] ALTER COLUMN [PhoneNumber] ADD MASKED WITH (FUNCTION = 'partial(1,"XXXXXXX",0)')

Access	Definition
SELECT	Can't view rows that are filtered.
UPDATE	Can't update rows that are filtered.
DELETE	Can't delete rows that are filtered.
INSERT	Not applicable.

Access	Definition
AFTER INSERT	Prevents users from inserting rows with values that violate the predicate.
AFTER UPDATE	Prevents users from updating rows to values that violate the predicate.
BEFORE UPDATE	Prevents users from updating rows that currently violate the predicate.
BEFORE DELETE	Blocks delete operations if the row violates the predicate.

Alerts	Definition
Vulnerability to SQL injection	This alert looks for T-SQL code coming into your database that may be vulnerable to SQL injection attacks. An example would be a stored procedure call that didn't sanitize user inputs.
Potential SQL injection	This alert is triggered when an attacker is actively attempting to execute a SQL injection attack.
Access from unusual location	This alert is triggered when a user logs in from an unusual geographic location.
Access from unusual Azure data center	This alert is looking for attacks from an Azure data center that isn't normally accessed.
Access from unfamiliar principal	This alert is raised when a user or applications log on to a database that they haven't previously accessed.
Access from a potentially harmful application	This alert detects common tools that are used to attack databases.
Brute force SQL credentials	This alert is triggered when there a high number of log in failures with different credentials.

Summary

- **1 minute**
- This module explored the practices of setting compliance controls on the data that is stored within the SQL Server database.
- You also reviewed several security features available for Azure SQL, including the Microsoft Defender for SQL.
- Now that you've reviewed this module, you should be able to:
- How data should be classified
- Why server and database audit are important
- How to implement row level security and dynamic data masking
- Understand the usage of Microsoft Defender for SQL

- **How Ledger works**
- Explore Microsoft Purview supported capabilities